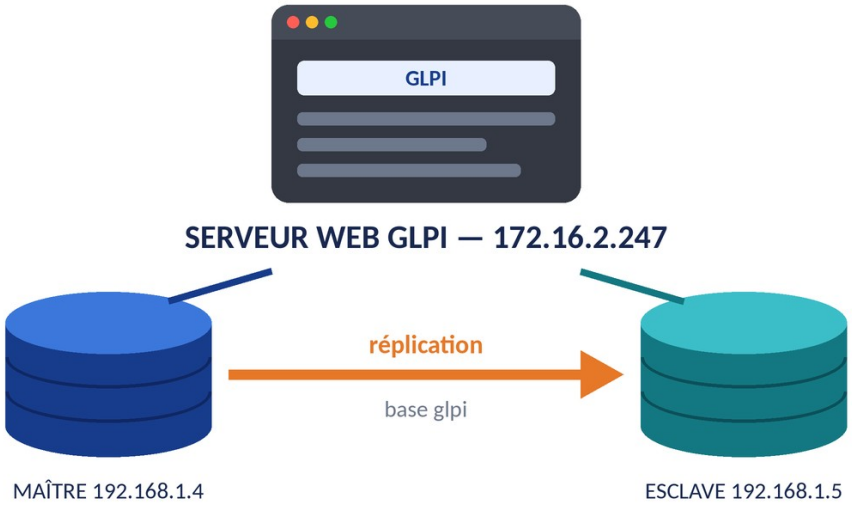
Documentation GLPI

Installation de GLPI 11 avec base de données déportée et répliquée — Projet M2L



Produit par Martinière Julien

À quoi sert GLPI ?

GLPI est une application web open source de **gestion de parc informatique** et de **service d'assistance**. Elle permet d'inventorier le matériel et les logiciels d'une organisation, de suivre les incidents et les demandes des utilisateurs sous forme de tickets, et de garder l'historique des interventions. Toutes ces informations sont stockées dans une base de données MariaDB.

SOMMAIRE :

1/ Contexte..... 3
Objectif..... 3
2/ Architecture..... 4
Réseau et flux..... 4
3/ Prérequis..... 5
4/ Étapes d'installation..... 6
Étape 1 : préparation des serveurs MariaDB..... 6
Étape 2 : droits administrateur sur la VM glpi..... 6
Étape 3 : installation d'Apache, PHP et du client MariaDB..... 7
Étape 4 : test de connexion aux bases..... 7
Étape 5 : téléchargement de GLPI 11.0.11..... 8
Étape 6 : configuration d'Apache et de PHP..... 8
Étape 7 : prérequis et installation de la base..... 9
Étape 8 : déclaration de l'esclave dans GLPI..... 9
Principe de fonctionnement..... 10
5/ Tests de validation..... 11
6/ Incidents rencontrés et résolution..... 12
Incident 1 : port 3306 injoignable..... 12
Incident 2 : sudo absent, puis demandant un mot de passe..... 12
Incident 3 : erreur HTTP 500 après l'installation..... 12
Incident 4 : retard de réplication de 3600 s..... 12
7/ Points restants..... 13

1/ Contexte

Le projet se déroule dans le contexte de la **M2L (Maison des Ligues)**.

Objectif

Installer GLPI sur une **VM dédiée au serveur web**. La base de données est hébergée sur **deux autres VM MariaDB**, en réplication maître/esclave :

- ✓ le **maître** reçoit les écritures ;
- ✓ l'**esclave** sert les lectures.

> **Note sur les mots de passe**
>
> Le mot de passe du compte `glpi` est masqué et remplacé par `<MDP_GLPI>`.
>
> Il s'agit d'un identifiant de **laboratoire**, utilisé uniquement en environnement de formation.

2/ Architecture

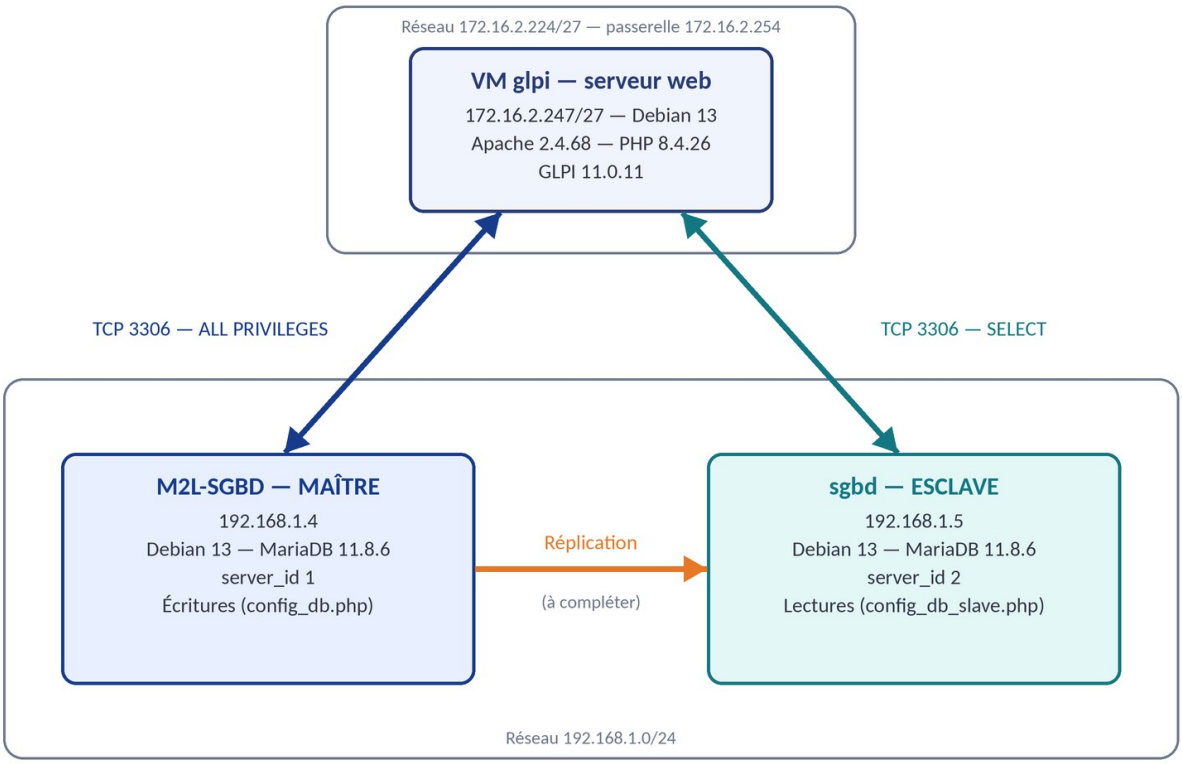


Figure 1 – Architecture GLPI avec base déportée et répliquée

Machine	Rôle	IP	Système
glpi	Serveur web GLPI (Apache + PHP), VM KVM Proxmox	172.16.2.247/27	Debian 13
M2L-SGBD	MariaDB maître, server_id 1 (écritures)	192.168.1.4	Debian 13, MariaDB 11.8.6
sgbd	MariaDB esclave, server_id 2 (lectures)	192.168.1.5	Debian 13, MariaDB 11.8.6

Réseau et flux

- ✓ Passerelle de la VM glpi : **172.16.2.254**. Elle assure le routage entre 172.16.2.224/27 et 192.168.1.0/24.
- ✓ Flux : VM glpi → 192.168.1.4 et 192.168.1.5 en **TCP 3306**.

3/ Prérequis

Élément	Valeur
Serveur web	VM Debian 13 `glpi` (172.16.2.247/27)
Bases de données	Deux VM Debian 13 avec MariaDB 11.8.6 (maître et esclave)
Base	`glpi` (utf8mb4)
Réseau	Routage entre 172.16.2.224/27 et 192.168.1.0/24, TCP 3306 autorisé
Version de GLPI	11.0.11

> **Réplication MariaDB**
> La mise en place de la réplication elle-même n'est pas détaillée ici : **à compléter**.

4/ Étapes d'installation

Étape 1 : préparation des serveurs MariaDB

Machine	Objectif
M2L-SGBD (maître) et sgbd (esclave)	Ouvrir MariaDB au réseau et créer le compte `glpi` sur chaque serveur.

Sur les deux serveurs, `bind-address = 0.0.0.0` est placé dans `/etc/mysql/mariadb.conf.d/50-server.cnf`, puis MariaDB est redémarré. La base s'appelle `glpi` (utf8mb4).

Sur le maître : le compte `glpi` reçoit tous les droits sur la base.

root@M2L-SGBD : préparation du maître

```
root@M2L-SGBD:~# nano /etc/mysql/mariadb.conf.d/50-server.cnf
#   bind-address = 0.0.0.0
root@M2L-SGBD:~# systemctl restart mariadb
root@M2L-SGBD:~# mariadb
MariaDB [(none)]> CREATE USER IF NOT EXISTS 'glpi'@'172.16.2.247' IDENTIFIED BY '<MDP_GLPI>';
MariaDB [(none)]> GRANT ALL PRIVILEGES ON glpi.* TO 'glpi'@'172.16.2.247';
MariaDB [(none)]> FLUSH PRIVILEGES;
```

Figure 2 – Préparation du maître M2L-SGBD

Sur l'esclave : le même compte reçoit seulement `SELECT` (lecture seule).

root@sgbd : préparation de l'esclave

```
root@sgbd:~# nano /etc/mysql/mariadb.conf.d/50-server.cnf
#   bind-address = 0.0.0.0
root@sgbd:~# systemctl restart mariadb
root@sgbd:~# mariadb
MariaDB [(none)]> CREATE USER IF NOT EXISTS 'glpi'@'172.16.2.247' IDENTIFIED BY '<MDP_GLPI>';
MariaDB [(none)]> GRANT SELECT ON glpi.* TO 'glpi'@'172.16.2.247';
MariaDB [(none)]> FLUSH PRIVILEGES;
-- lecture seule : SELECT uniquement
```

Figure 3 – Préparation de l'esclave sgbd

> **Résultat attendu**
>
> MariaDB écoute sur toutes les interfaces. Le compte `'glpi'@'172.16.2.247'` existe sur les deux serveurs.

Étape 2 : droits administrateur sur la VM glpi

Machine	Objectif
glpi (en root)	Donner temporairement les droits sudo à l'utilisateur `glpi`.

root@glpi : droits administrateur temporaires

```
root@glpi:~# apt install -y sudo && usermod -aG sudo glpi
root@glpi:~# echo 'glpi ALL=(ALL) NOPASSWD:ALL' > /etc/sudoers.d/glpi \
    && chmod 440 /etc/sudoers.d/glpi
# fichier /etc/sudoers.d/glpi supprimé une fois l'installation terminée
```

Figure 4 – Installation de sudo et règle temporaire

Résultat attendu

L'utilisateur `glpi` peut lancer `sudo` sans mot de passe pendant l'installation.

Le fichier `/etc/sudoers.d/glpi` a été **supprimé** une fois l'installation terminée.

Étape 3 : installation d'Apache, PHP et du client MariaDB

Machine	Objectif
glpi	Installer le serveur web, PHP et ses extensions, et le client MariaDB.

glpi@glpi : Apache, PHP et client MariaDB

```
glpi@glpi:~$ sudo apt-get update
glpi@glpi:~$ sudo apt-get install -y apache2 libapache2-mod-php php php-mysql \
    php-curl php-gd php-intl php-xml php-mbstring php-zip php-bz2 php-ldap \
    php-bcmath php-opcache php-apcu mariadb-client
# versions obtenues : Apache 2.4.68 — PHP 8.4.26
# pas de serveur MariaDB sur cette VM : client uniquement
```

Figure 5 – Installation des paquets

Résultat attendu

Apache **2.4.68** et PHP **8.4.26** installés.

Pas de serveur MariaDB sur cette VM : seulement le client.

Étape 4 : test de connexion aux bases

Machine	Objectif
glpi	Vérifier que la VM glpi atteint les deux serveurs avec le bon niveau de droits.

glpi@glpi : test de connexion aux deux bases

```
glpi@glpi:~$ mariadb --skip-ssl -h 192.168.1.4 -u glpi -p'<MDP_GLPI>' \
    -e "SHOW DATABASES; SHOW GRANTS;"
# → connexion OK — base glpi visible — ALL PRIVILEGES sur glpi.*
glpi@glpi:~$ mariadb --skip-ssl -h 192.168.1.5 -u glpi -p'<MDP_GLPI>' \
    -e "SHOW DATABASES; SHOW GRANTS;"
# → connexion OK — base glpi visible — SELECT sur glpi.*
```

Figure 6 – Connexion au maître et à l'esclave

Résultat attendu

Connexion OK sur les deux serveurs. Base `glpi` visible.

ALL PRIVILEGES sur le maître, **SELECT** sur l'esclave.

Étape 5 : téléchargement de GLPI 11.0.11

Machine	Objectif
glpi	Récupérer GLPI et le placer dans `/var/www`.

La version **11.0.11** a été choisie. La 12.0.0 venait de sortir le 7 octobre 2026 et a été jugée trop récente.

glpi@glpi : téléchargement de GLPI 11.0.11

```
glpi@glpi:~$ wget https://github.com/glpi-project/glpi/releases/download/\
    11.0.11/glpi-11.0.11.tgz
glpi@glpi:~$ sudo tar -xzf glpi-11.0.11.tgz -C /var/www/
glpi@glpi:~$ sudo chown -R www-data:www-data /var/www/glpi
```

Figure 7 – Téléchargement et extraction de GLPI

> **Résultat attendu**
>
> GLPI est extrait dans `/var/www/glpi`, propriété de `www-data`.

Étape 6 : configuration d'Apache et de PHP

Machine	Objectif
glpi	Publier GLPI via un site Apache dédié et régler PHP.

Le site Apache pointe sur le dossier `public` de GLPI. Les règles de réécriture renvoient les requêtes vers `index.php` et transmettent l'en-tête `Authorization`.

/etc/apache2/sites-available/glpi.conf

```
glpi@glpi:~$ cat /etc/apache2/sites-available/glpi.conf
<VirtualHost *:80>
    ServerName glpi
    DocumentRoot /var/www/glpi/public
    <Directory /var/www/glpi/public>
        Require all granted
        RewriteEngine On
        RewriteCond %{HTTP:Authorization} ^(.+)$
        RewriteRule .* - [E=HTTP_AUTHORIZATION:%{HTTP:Authorization}]
        RewriteCond %{REQUEST_FILENAME} !-f
        RewriteRule ^(.*)$ index.php [QSA,L]
    </Directory>
    ErrorLog ${APACHE_LOG_DIR}/glpi_error.log
    CustomLog ${APACHE_LOG_DIR}/glpi_access.log combined
</VirtualHost>
```

Figure 8 – Site Apache glpi.conf

Le fichier PHP `glpi.ini` règle la mémoire, la durée d'exécution, la taille des envois, la sécurité des cookies de session et le fuseau horaire.

/etc/php/8.4/mods-available/glpi.ini

```
glpi@glpi:~$ cat /etc/php/8.4/mods-available/glpi.ini
memory_limit = 256M
max_execution_time = 600
upload_max_filesize = 20M
post_max_size = 20M
session.cookie_httponly = On
session.cookie_samesite = Lax
date.timezone = Europe/Paris
```

Figure 9 – Réglages PHP glpi.ini

On active ensuite la configuration PHP, le module `rewrite` et le site `glpi`, en désactivant le site par défaut.

glpi@glpi : activation de la configuration

```
glpi@glpi:~$ sudo phpenmod glpi
glpi@glpi:~$ sudo a2enmod rewrite
glpi@glpi:~$ sudo a2dissite 000-default
glpi@glpi:~$ sudo a2ensite glpi
glpi@glpi:~$ sudo apache2ctl configtest
glpi@glpi:~$ sudo systemctl restart apache2
```

Figure 10 – Activation et redémarrage d'Apache

Résultat attendu

Apache redémarre avec le site `glpi` actif.

Étape 7 : prérequis et installation de la base

Machine	Objectif
glpi	Vérifier les prérequis puis créer le schéma GLPI sur le maître.

glpi@glpi : prérequis et installation de la base

```
glpi@glpi:~$ cd /var/www/glpi
glpi@glpi:/var/www/glpi$ sudo -u www-data php bin/console system:check_requirements
# → tous les prérequis sont OK
glpi@glpi:/var/www/glpi$ sudo -u www-data php bin/console db:install -H 192.168.1.4 \
    -d glpi -u glpi -p '<MDP_GLPI>' -L fr_FR --no-telemetry --no-interaction
# → 442 tables créées sur le maître, répliquées sur l'esclave
# → génère config/config_db.php (pointe sur le maître 192.168.1.4)
```

Figure 11 – Vérification des prérequis et installation de la base

Résultat attendu

Tous les prérequis sont OK.

442 tables créées sur le maître et répliquées sur l'esclave.

Le fichier `/var/www/glpi/config/config_db.php` est généré. Il pointe sur le maître 192.168.1.4.

Étape 8 : déclaration de l'esclave dans GLPI

Machine	Objectif
glpi	Indiquer à GLPI le serveur à utiliser pour les lectures.

/var/www/glpi/config/config_db_slave.php

```
glpi@glpi:/var/www/glpi$ cat config/config_db_slave.php
<?php
class DBSlave extends DBmysql {
   public $slave = true;
   public $dbhost = '192.168.1.5';
   public $dbuser = 'glpi';
   public $dbpassword = '<MDP_GLPI>';
   public $dbdefault = 'glpi';
   public $use_utf8mb4 = true;
   public $allow_datetime = false;
   public $allow_signed_keys = false;
}
```

Figure 12 – Fichier config_db_slave.php

glpi@glpi : droits et suppression de l'installeur

```
glpi@glpi:~$ sudo chown www-data:www-data /var/www/glpi/config/config_db_slave.php
glpi@glpi:~$ sudo chmod 640 /var/www/glpi/config/config_db.php \
    /var/www/glpi/config/config_db_slave.php
glpi@glpi:~$ sudo rm -f /var/www/glpi/install/install.php
```

Figure 13 – Droits des fichiers et suppression de l'installeur

Principe de fonctionnement

Fichier	Serveur	Usage
`config_db.php`	Maître 192.168.1.4	Écritures
`config_db_slave.php`	Esclave 192.168.1.5	Lectures

Si le maître tombe, GLPI **bascule en lecture seule** sur l'esclave.

5/ Tests de validation

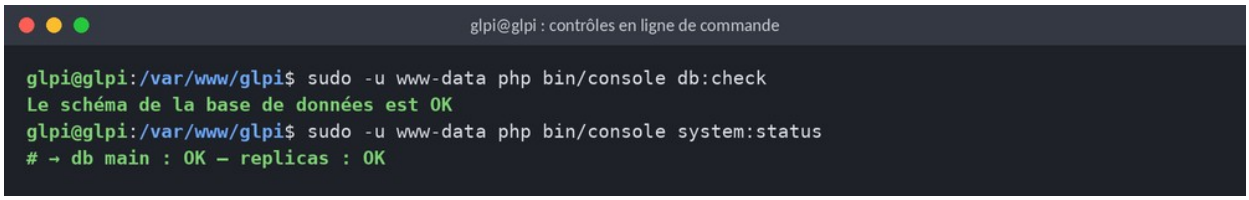


Figure 14 – Contrôles en ligne de commande

Test	Machine	Résultat attendu	Résultat obtenu
Page d'accueil http://172.16.2.247	Poste client	Page « Authentification - GLPI »	Affichée, HTTP 200 ✔️
Connexion glpi / glpi, source « Base interne GLPI »	Poste client	Accès à l'interface	Interface standard ✔️
`db:check`	glpi	Schéma OK	« Le schéma de la base de données est OK » ✔️
`system:status`	glpi	Base principale et réplica OK	db main OK, replicas OK ✔️
Nombre de tables	Maître et esclave	Identique	442 des deux côtés ✔️
Dernier id dans `glpi_logs`	Maître et esclave	Identique	Identique ✔️

> **« Base interne GLPI » : de quoi s'agit-il ?**
> C'est la **source d'authentification** : les comptes de la table `glpi_users`, stockée sur le SGBD.
> Ce n'est **pas** le choix du serveur de base de données.

> **Capture d'écran**
> [Capture d'écran : page « Authentification - GLPI » sur http://172.16.2.247]

> **Capture d'écran**
> [Capture d'écran : interface standard de GLPI après connexion]

> **Capture d'écran**
> [Capture d'écran : résultat de system:status avec db main OK et replicas OK]

6/ Incidents rencontrés et résolution

Incident 1 : port 3306 injoignable

Cause	Résolution
MariaDB écoutait seulement sur 127.0.0.1.	`bind-address = 0.0.0.0` et création de l'utilisateur distant (étape 1).

Incident 2 : sudo absent, puis demandant un mot de passe

Cause	Résolution
sudo n'était pas installé sur la VM glpi.	Installation de sudo et règle NOPASSWD temporaire (étape 2).

Incident 3 : erreur HTTP 500 après l'installation

Message affiché : « The parameter "kernel.secret" has a dependency on a non-existent parameter ».

Cause	Résolution
La clé `/var/www/glpi/config/glpicrypt.key`, générée aléatoirement, contenait deux caractères `%`. Symfony les interprétait comme une variable.	Aucune donnée n'étant encore chiffrée, régénération d'une clé de 32 octets sans `%`, puis vidage du cache.



Figure 15 – Sauvegarde, régénération de la clé et vidage du cache

Incident 4 : retard de réplication de 3600 s

Cause	Résolution
GLPI affichait 3600 s de retard alors que les données étaient identiques. L'esclave était en heure de Londres (BST), le maître en heure de Paris (CEST).	Passage de l'esclave en Europe/Paris, puis redémarrage de MariaDB.

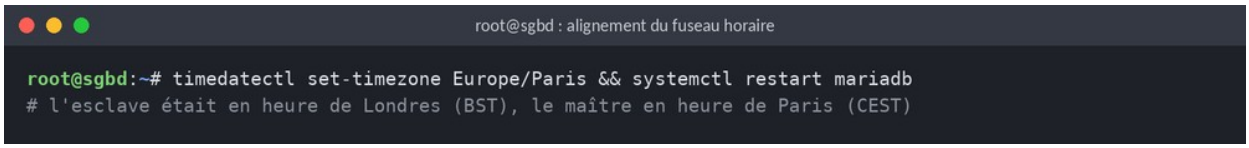


Figure 16 – Alignement du fuseau horaire de l'esclave

7/ Points restants

Point	Détail	État
Esclave en lecture seule	Passer `read_only = 1` dans la section `[mysqld]` de `50-server.cnf` sur l'esclave. Encore à OFF.	À faire
Comptes par défaut	Changer les mots de passe de `glpi`, `tech`, `normal` et `post-only`.	À faire
Sauvegarde de clé	Supprimer `/var/www/glpi/config/glpicrypt.key.bak-pourcent`.	À faire
Actions automatiques	Tâche cron des actions automatiques de GLPI.	Non réalisé
HTTPS	Accès chiffré à GLPI.	Non réalisé

> **Sécurité**
>
> Les comptes par défaut (`glpi`, `tech`, `normal`, `post-only`) doivent être sécurisés avant toute mise en production.